How Should a Neural Network Take a Step? Muon and Matrix Geometry

N-20261017

A neural network has millions or billions of numbers to adjust. It is tempting to think that an optimizer merely decides how far to move each one. But those numbers do not live in isolation. Many form matrices, and a matrix acts on an entire space of activations. The question of how large an update is already has several quite different answers.

This becomes particularly vivid when we compare AdamW and Muon. Muon keeps only a momentum state for the hidden weight matrices it handles, often makes rapid progress early in language-model pretraining, and yet does not always produce a substantially better model at the end. The practical gain may be enough to prefer it. The mathematical puzzle is more interesting: *what aspect of the geometry of a neural network does Muon manage to respect?*

We will begin with two numbers, pass through matrix duality and the polar decomposition, and then return to the whole Transformer. Throughout, a good update is good *relative to a specified norm, local objective, and budget.* Choosing those three ingredients is precisely the difficulty.[1]

§1.1 Two directions hidden in a tiny gradient

Suppose the gradient of a loss with respect to a 2×2 weight matrix is

$$G = \begin{pmatrix} 8 & 0 \\ 0 & 1 \end{pmatrix}.$$

The first diagonal entry asks for an update eight times as large as the second one. For now, forget all momentum and training noise and consider two possible unit-scale steps:

$$D_{\mathrm{grad}} = -\frac{1}{8}\begin{pmatrix} 8 & 0 \\ 0 & 1 \end{pmatrix} = -\begin{pmatrix} 1 & 0 \\ 0 & 1/8 \end{pmatrix}, \qquad D_{\mathrm{polar}} = -\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

Both have largest singular value 1. The first respects the relative size of the two gradient entries. The second gives the two singular directions equal strength.

To first order, the predicted loss change is $\langle G, D \rangle = \mathrm{tr}(G^\top D)$. Hence

$$\langle G, D_{\mathrm{grad}} \rangle = -8 - \frac{1}{8} = -\frac{65}{8}, \qquad \langle G, D_{\mathrm{polar}} \rangle = -9.$$

With the *same upper bound on the spectral norm*, the second step achieves a larger first-order decrease. But its Frobenius length is $\sqrt{2}$, whereas that of the first step is only $\sqrt{65}/8$. We must not quietly switch from one notion of size to another halfway through a comparison.

[1] For the algorithms and scaling observations, see Keller Jordan and collaborators, *Muon: An optimizer for hidden layers in neural networks* (2024), and Jingyuan Liu and collaborators, *Muon is Scalable for LLM Training* (2025). The sources describe related but distinct practical recipes.

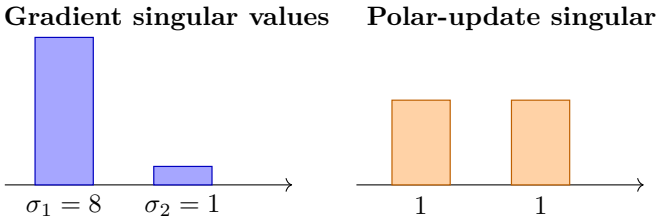


Figure 1.1: The polar factor preserves the singular directions while flattening the nonzero singular values. Bar heights are schematic; labels give the exact values.

The example poses a useful question. Is it better to keep the information in the gradient magnitudes, or to prevent a few large singular modes from swallowing nearly the entire update? Neither preference makes sense until we decide what constitutes a unit step.

§1.2 A norm tells us how to spend a step

For a differentiable loss $L(W)$, a first-order approximation near W gives

$$L(W + D) = L(W) + \langle G, D\rangle + o(\|D\|), \qquad G = \nabla_W L(W).$$

Fix a radius $\eta > 0$ and a matrix norm $\|\cdot\|_{\mathcal{X}}$. The associated first-order update solves

$$\min_{\|D\|_{\mathcal{X}} \le \eta} \langle G, D\rangle. \tag{1.1}$$

Geometrically, we slide a hyperplane perpendicular to G across a norm ball until it first touches the ball.

In two dimensions, a Euclidean ball is round, while an ℓ^∞ ball is a square. Their supporting points generally lie in different directions. The corners of the square make it possible to move at full speed in several coordinates at once.

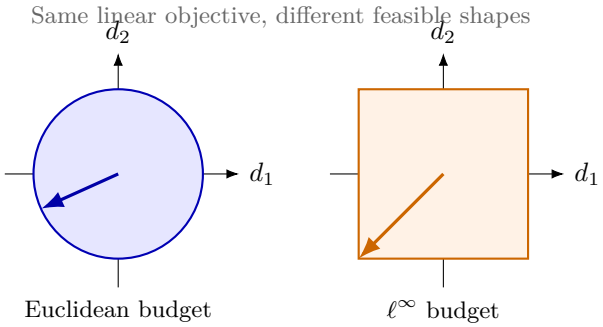


Figure 1.2: A unit ball is part of the definition of an optimizer's geometry. The arrows illustrate minimizing directions for a gradient with positive, unequal coordinates; the drawing is schematic.

To state the calculation precisely, the *dual norm* is

$$\|G\|_{\mathcal{X},*} = \max_{\|D\|_{\mathcal{X}} \le 1} \langle G, D\rangle.$$

Symmetry of a norm ball immediately gives

$$\min_{\|D\|_{\mathcal{X}} \le \eta} \langle G, D \rangle = -\eta \|G\|_{\mathcal{X},*}. \tag{1.2}$$

The value comes from the dual norm; a maximizing vector determines the direction. This distinction matters when a practical optimizer discards the dual-norm magnitude and prescribes a learning rate instead.

For a vector gradient g, the standard cases are easy to check:

Budget on d	Optimal direction	First-order reduction per unit radius
$\|d\|_2 \le 1$	$-g/\|g\|_2$	$\|g\|_2$
$\|d\|_\infty \le 1$	$-\operatorname{sign}(g)$	$\|g\|_1$
$\|d\|_1 \le 1$	$-e_j \operatorname{sign}(g_j),\ j \in \arg\max_i \lvert g_i \rvert$	$\|g\|_\infty$

Thus the familiar $\ell^1, \ell^2, \ell^\infty$ geometries make three different decisions: concentrate the budget, preserve the gradient direction, or update all coordinates at full permitted magnitude. The formula for ℓ^2 assumes $g \neq 0$; at $g = 0$ every feasible update minimizes the linear term.

One can also replace the hard constraint by

$$\min_D \left\{ \langle G, D \rangle + \frac{\lambda}{2} \|D\|_{\mathcal{X}}^2 \right\}, \qquad \lambda > 0.$$

If D_1 is a unit-ball minimizer and $G \neq 0$, the optimum has the form

$$D_\lambda = \frac{\|G\|_{\mathcal{X},*}}{\lambda} D_1. \tag{1.3}$$

Indeed, any $D = rT$ with $\|T\|_{\mathcal{X}} = 1$ reduces the problem to minimizing $-r\|G\|_{\mathcal{X},*} + \lambda r^2/2$. Choosing a *direction* by a norm and choosing its *scale* by a learning-rate schedule are separable modeling decisions.[2]

§1.3 Which two-norm? A matrix has several

We should settle a terminological trap before going further. For a vector, $\|x\|_2$ has an unambiguous meaning. For a matrix $A \in \mathbb{R}^{m \times n}$, both of these are common:

$$\|A\|_{\mathrm{F}} = \left(\sum_{i,j} A_{ij}^2 \right)^{1/2}, \qquad \|A\|_{\mathrm{op}} = \max_{x \neq 0} \frac{\|Ax\|_2}{\|x\|_2}.$$

The first treats a matrix as a long vector of mn entries. The second treats it as an operator on input activations. Some authors write $\|A\|_2$ for the *second* quantity, even though the first is the Schatten-2 norm.

If $A = U \operatorname{diag}(\sigma_1, \dots, \sigma_r) V^\top$ is a compact singular value decomposition, then

$$\|A\|_{\mathrm{op}} = \max_i \sigma_i, \qquad \|A\|_{\mathrm{F}} = \Big(\sum_i \sigma_i^2 \Big)^{1/2}, \qquad \|A\|_* = \sum_i \sigma_i.$$

Here $\|\cdot\|_*$ is the nuclear or trace norm. For the matrix of all ones in dimension n,

$$\|A\|_{\mathrm{op}} = n, \qquad \|A\|_{\mathrm{F}} = n, \qquad \|A\|_* = n;$$

[2] The geometric viewpoint follows Jeremy Bernstein and Laker Newhouse, *Old Optimizer, New Norm: An Anthology* (2024). The steepest-descent identities used here also follow directly from finite-dimensional norm duality.

but for the identity I_n,

$$\|I_n\|_{\mathrm{op}} = 1, \qquad \|I_n\|_{\mathrm{F}} = \sqrt{n}, \qquad \|I_n\|_* = n.$$

The identity has no particularly large effect on any *single* input direction. Nevertheless, it acts nontrivially on *all* n directions, a distinction detected by the Frobenius and nuclear norms.

These quantities are special cases of the Schatten norms

$$\|A\|_{S_p} = \left(\sum_i \sigma_i^p\right)^{1/p}, \quad 1 \le p < \infty, \qquad \|A\|_{S_\infty} = \max_i \sigma_i.$$

In particular, S_2 is the Frobenius norm and S_∞ is the spectral norm. The phrase about learning in a Euclidean two-norm rather than a spectral norm only becomes meaningful once this distinction is fixed.

§1.4 Why spectral steepest descent is a matrix sign

The dual norm of the spectral norm is the nuclear norm. An especially transparent proof also gives the update direction.

Proposition 1.4.1

Let $G \in \mathbb{R}^{m\times n}$ have compact SVD $G = U_r \operatorname{diag}(\sigma_1, \dots, \sigma_r) V_r^\top$, where $\sigma_i > 0$. Then

$$\min_{\|D\|_{\mathrm{op}} \le \eta} \langle G, D\rangle = -\eta \sum_{i=1}^{r} \sigma_i,$$

and a minimizer is

$$D_{\mathrm{sp}} = -\eta U_r V_r^\top. \tag{1.4}$$

Proof. Write u_i, v_i for the singular vectors. For every admissible D,

$$\langle G, D\rangle = \sum_i \sigma_i\, u_i^\top D v_i \ge -\sum_i \sigma_i \|D v_i\|_2 \ge -\eta \sum_i \sigma_i.$$

For $D = -\eta U_r V_r^\top$, the identity $U_r^\top U_r = V_r^\top V_r = I_r$ shows $Dv_i = -\eta u_i$, so both inequalities are equalities. □

The matrix $U_r V_r^\top$ is the *partial polar factor*, also called the matrix sign in this context. All its nonzero singular values are 1. For an invertible square matrix, it is orthogonal; for a rectangular matrix, it has orthonormal columns or rows on the relevant subspace. There is no requirement that the weight matrix itself become orthogonal.

Contrast this with the Frobenius-constrained solution:

$$D_{\mathrm{F}} = -\eta \frac{G}{\|G\|_{\mathrm{F}}}.$$

The spectral optimizer keeps the singular vectors and flattens the singular values; the Frobenius optimizer keeps the ratios between singular values. Both are exact optimizers of their respective *linearized* problems.

We also have a precise answer to the opening calculation. For $G = \operatorname{diag}(8, 1)$, spectral descent gives $-\eta I_2$, while Frobenius descent gives $-\eta \operatorname{diag}(8, 1)/\sqrt{65}$. The gradient-scaled step of our opening example used a *spectral*, rather than Frobenius, radius. Comparing the three objects prevents two meanings of a normalized gradient from being conflated.

§1.5 Between preserving and erasing the spectrum

The difference between those two updates suggests that the choice need not be all-or-nothing. For $1 < p < \infty$, write $q = p/(p-1)$. Schatten-p and Schatten-q norms are dual, just as vector ℓ^p and ℓ^q norms are dual.

Proposition 1.5.1

Under the constraint $\|D\|_{S_p} \le \eta$, one optimal update is

$$D_p = -\eta U_r \operatorname{diag}\left(\frac{\sigma_i^{q-1}}{(\sum_j \sigma_j^q)^{1/p}}\right)_{i=1}^{r} V_r^\top. \tag{1.5}$$

Its first-order decrease is $-\eta\|G\|_{S_q}$.

Proof. The singular-value inequality gives $\langle G, D\rangle \ge -\sum_i \sigma_i s_i(D)$. By Hölder's inequality,

$$\sum_i \sigma_i s_i(D) \le \left(\sum_i \sigma_i^q\right)^{1/q} \left(\sum_i s_i(D)^p\right)^{1/p}.$$

Equality holds when the singular vectors are aligned in opposite directions and $s_i(D)$ is proportional to σ_i^{q-1}. Setting the p-norm of that singular-value vector to η gives the stated normalization. □

It is illuminating to suppress the overall scalar and look only at

$$\Psi_a(G) = U_r \operatorname{diag}(\sigma_i^a) V_r^\top, \qquad 0 \le a \le 1.$$

When $a = 1$, the spectral magnitudes are preserved. When $a = 0$, every nonzero singular value becomes 1. Intermediate exponents gently compress the dynamic range. More precisely, the *direction* $\Psi_a(G)$ is that of Schatten-p steepest descent for $p = 1 + 1/a$, when $a > 0$; the limit $a \to 0$ corresponds to S_∞.

For our two-singular-value example:

a	relative singular magnitudes	ratio of large to small
1	$(8, 1)$	8
1/2	$(\sqrt{8}, 1)$	$\sqrt{8}$
1/4	$(8^{1/4}, 1)$	$8^{1/4}$
0	$(1, 1)$	1

This is a family of *directions*. A practical implementation must still say how the direction is normalized and how the step size is selected.

There is an important caveat. Applying Ψ_a to a momentum estimate, or to an Adam-normalized update, is not generally the same as solving the original loss's steepest-descent problem in a single fixed Schatten geometry. The transformed input already depends on optimization history. The spectral family remains useful precisely because it lets us study *how much* of that input's spectrum should be kept.

§1.6 AdamW lives in moving coordinates

Let g_t be the stochastic gradient at iteration t; write all products and divisions below entrywise. Adam maintains

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1) g_t, \\ v_t &= \beta_2 v_{t-1} + (1-\beta_2) g_t \odot g_t, \\ \widehat{m}_t &= \frac{m_t}{1-\beta_1^t}, \qquad \widehat{v}_t = \frac{v_t}{1-\beta_2^t}. \end{aligned}$$

With decoupled weight decay, the update is

$$W_{t+1} = (1-\eta_t\lambda) W_t - \eta_t \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon}. \tag{1.6}$$

The first state tracks a preferred direction; the second tracks the recent size of gradient coordinates. Coordinates with large historical squared gradients are damped.

It would be inaccurate to say that full AdamW is simply Euclidean gradient descent. Freeze v_t, set $D_t = \operatorname{diag}((\sqrt{\widehat{v}_{t,i}} + \varepsilon)^{-1})$ after vectorizing the matrix, and ignore weight decay. The update is then

$$d_t = -\eta_t D_t \widehat{m}_t.$$

This is the minimizer of

$$\min_d \left\{ \widehat{m}_t^\top d + \frac{1}{2\eta_t} d^\top D_t^{-1} d \right\}. \tag{1.7}$$

Thus, at a frozen instant, Adam behaves like steepest descent in a *weighted quadratic geometry*. The geometry moves with v_t; the momentum $\widehat{m}_t$ is also generally different from the instantaneous gradient. Equation (1.7) is an interpretation of the direction conditional on these states, not an exact static-norm description of the entire Adam trajectory.

There is a second, simpler connection. If $\beta_1 = \beta_2 = 0$ and $\varepsilon = 0$ on nonzero gradient coordinates, Adam's update direction is

$$\frac{g_t}{\sqrt{g_t \odot g_t}} = \operatorname{sign}(g_t).$$

This is the ℓ^∞-budget direction from our norm-ball picture. With realistic momentum and second-moment estimates, however, Adam's direction need not be a sign vector.

The distinction is both mathematically and experimentally important. Muon and AdamW do not form a clean opposition between spectral norm and ordinary L^2. A cleaner comparison is between *spectral normalization of a matrix-valued momentum* and *history-dependent coordinatewise preconditioning*. Their norm geometries are related but their actual optimization procedures contain additional ingredients.

§1.7 Muon orthogonalizes a momentum matrix

For a hidden matrix W_t, a typical Muon step maintains one momentum buffer

$$M_t = \beta M_{t-1} + G_t$$

and forms a Nesterov-style combination such as $\widetilde{M}_t = \beta M_t + G_t$. It then computes an approximation to the polar factor

$$\operatorname{polar}(\widetilde{M}_t) = U_r V_r^\top$$

and uses a shape-dependent scale $s_{m,n}$:

$$W_{t+1} = (1 - \eta_t \lambda) W_t - \eta_t s_{m,n} \operatorname{polar}(\widetilde{M}_t). \tag{1.8}$$

Momentum conventions, the exact normalization of the gradient, weight decay and scaling factors vary across implementations. It is the *polar operation on a matrix update* that identifies the family.

Why not compute a full SVD at every step? Muon uses a small number of Newton–Schulz-type matrix iterations. Starting with a normalized matrix X_0, a popular form applies a polynomial

$$X_{k+1} = aX_k + b(X_k X_k^\top) X_k + c(X_k X_k^\top)^2 X_k. \tag{1.9}$$

If $X_k = U \operatorname{diag}(s_i) V^\top$, the next iterate has the same singular vectors and transformed singular values

$$s_i \longmapsto a s_i + b s_i^3 + c s_i^5.$$

We can think of the iteration as repeatedly redrawing the *heights* of the singular-value bars, ideally pushing them toward a common level. The real computation uses matrix multiplications and avoids explicitly extracting those bars.

A finite number of polynomial iterations only *approximates* the polar factor. The approximation can also matter for small singular values; the different published coefficient choices and scaling rules should not be treated as mathematically identical. In particular, the efficient GPU implementation is an algorithmic achievement in its own right, distinct from the abstract solution of a spectral-norm trust-region problem.

The standard practice is hybrid: Muon handles selected two-dimensional hidden weights, while embeddings, normalization scales, biases and frequently the output head remain on AdamW. A Transformer therefore uses more than one update geometry at the same time.

The polar step also does not make second-moment information intrinsically useless. For example, the 2026 preprint *Muon*[2] applies an Adam-style second-moment preconditioner *before* orthogonalization, partly to improve the conditioning of Newton–Schulz inputs. Its factorized variant trades additional state against cheaper approximations. This is a different algorithm from the one-state Muon discussed here, and is another reason to separate geometry from implementation.

§1.8 The memory advantage has an exact accounting

Suppose a hidden weight matrix contains $N = mn$ parameters and its optimizer states are stored in 32-bit floating-point numbers. Ignoring distributed sharding, the persistent state count for that matrix is

Optimizer on the matrix	Persistent state tensors	State bytes
AdamW	$m_t,\ v_t$	$8N$
Muon	M_t	$4N$

That is a genuine reduction of *one full state tensor* for eligible matrix parameters. It does not require any claim about validation loss to be valuable.

For a concrete scale, a matrix with $N = 10^8$ entries costs about 800 MB of such AdamW optimizer state and 400 MB of Muon momentum state (decimal megabytes). But this is deliberately a *state-only* calculation.

Real training also stores parameter values, gradients, possibly FP32 master weights, activations, communication buffers and allocator overhead. Muon's matrix iterations

require temporary workspace, and the nonmatrix parameters still use AdamW. ZeRO or tensor parallelism can change which rank holds the persistent state and how frequently it is communicated. Therefore the per-matrix state saving is certain under the stated storage assumptions; a particular reduction in *peak GPU memory* must be measured rather than inferred by multiplying the number of model parameters by four bytes.

A similar separation is needed for speed. Muon can use fewer optimization steps to reach a loss threshold and still take more time per step, because its orthogonalization involves extra matrix multiplications. The two comparisons to report are

$$\text{loss versus tokens seen}, \qquad \text{loss versus actual wall-clock time}.$$

The first measures sample efficiency; the second captures the computer on which one actually trains.

§1.9 Why rectangular matrices need different scales

Even a perfect polar factor has a surprisingly shape-dependent *entrywise* size. Let $P = U_r V_r^\top$ be the polar factor of a rank-r matrix of shape $m \times n$. Since the singular values of P are 1 on its active rank,

$$\|P\|_{\mathrm{F}}^2 = r.$$

Define the root-mean-square entry magnitude by $\mathrm{RMS}(P) = \|P\|_{\mathrm{F}}/\sqrt{mn}$. We obtain the exact identity

$$\mathrm{RMS}(P) = \sqrt{\frac{r}{mn}}. \tag{1.10}$$

For a full-rank matrix, $r = \min(m, n)$, so

$$\mathrm{RMS}(P) = \frac{1}{\sqrt{\max(m, n)}}. \tag{1.11}$$

A square $d \times d$ hidden matrix has polar-update RMS $1/\sqrt{d}$. A $d \times 4d$ expansion matrix has RMS $1/(2\sqrt{d})$. If we used the same raw multiplier for both, the rectangular layer would receive an update with half the RMS, even though the matrices are participating in the same network.

Multiplying by $\sqrt{\max(m, n)}$ aligns the full-rank RMS at 1. Liu and collaborators use a multiplier of the form $0.2\sqrt{\max(m, n)}$ to aim for an AdamW-like update RMS in their training recipe. The factor 0.2 is an *empirical scale choice*; the square-root dimension factor follows from the rank calculation.

Two cautions follow. The formula is exact for the ideal rank-r polar factor, while practical Newton–Schulz outputs have approximately flattened singular values. And equal RMS does not mean equal effect on the network. The latter issue takes us from the optimizer to the actual role of a matrix.

§1.10 A layer is a map, not a table of parameters

A hidden linear layer transforms an activation $x \in \mathbb{R}^{d_{\mathrm{in}}}$ into

$$y = Wx, \qquad W \in \mathbb{R}^{d_{\mathrm{out}} \times d_{\mathrm{in}}}.$$

If the parameters change by D, the layer output changes by

$$\delta y = Dx.$$

The spectral norm has a direct functional meaning:

$$\sup_{x\neq 0} \frac{\|Dx\|_2}{\|x\|_2} = \|D\|_{\mathrm{op}}. \tag{1.12}$$

Constraining $\|D\|_{\mathrm{op}}$ limits the worst amplification of an input through the update map. It is a plausible geometric prior for a linear layer whose inputs and outputs are measured by Euclidean norms.

This argument is suggestive, not conclusive. Embedding tables are typically used by *selecting rows*, not by multiplying arbitrary dense input vectors. Attention weights are embedded in bilinear interactions, a softmax, and residual connections. Normalization parameters act by coordinatewise multiplication. Treating every parameter block as an arbitrary linear operator throws away those different roles.

Even among linear maps, the notion of a unit step can depend on width. A vector RMS is

$$\|x\|_{\mathrm{rms}} = \frac{\|x\|_2}{\sqrt{d_{\mathrm{in}}}}, \qquad \|y\|_{\mathrm{rms}} = \frac{\|y\|_2}{\sqrt{d_{\mathrm{out}}}}.$$

The induced RMS-to-RMS operator norm is

$$\|D\|_{\mathrm{rms}\to\mathrm{rms}} = \sqrt{\frac{d_{\mathrm{in}}}{d_{\mathrm{out}}}}\,\|D\|_{\mathrm{op}}. \tag{1.13}$$

Thus a stable worst-case *RMS* effect across rectangular layers suggests a width correction. The appropriate factor depends on which spaces and normalization conventions we regard as physically meaningful. This is one reason that matrix shapes and parameterization rules must be part of a serious optimizer comparison.

§1.11 The same network can have very different coordinates

There is a simple obstruction to choosing an optimizer geometry from flattened parameters alone. Consider a two-layer linear network,

$$f(x) = W_2 W_1 x.$$

For any nonzero scalar c,

$$W_1' = cW_1, \qquad W_2' = c^{-1}W_2$$

produces exactly the same function. Yet an entrywise or Frobenius update with the same raw numerical scale can have a different effect in these new coordinates.

To see this without matrix notation, set $f(x) = abx$ with scalar parameters a, b. Let the loss depend on $z = ab$ through some differentiable function $\ell(z)$. Its parameter gradients are

$$\frac{\partial L}{\partial a} = b\ell'(ab), \qquad \frac{\partial L}{\partial b} = a\ell'(ab).$$

Under the equivalent parameterization $a' = ca,\ b' = b/c$, these become

$$\frac{\partial L}{\partial a'} = \frac{b}{c}\ell'(ab), \qquad \frac{\partial L}{\partial b'} = ca\ell'(ab).$$

Euclidean gradient descent therefore takes different *functional* steps in two descriptions of the same current model. Indeed, to first order,

$$\delta z = b\,\delta a + a\,\delta b,$$

and the two gradient steps induce

$$\delta z = -\eta(a^2 + b^2)\ell'(z), \qquad \delta z' = -\eta\left(c^2a^2 + \frac{b^2}{c^2}\right)\ell'(z).$$

For large c, the second expression may be arbitrarily larger. The ambiguity is already present in the simplest possible neural network.

This calculation does not prove that there exists one perfectly parameterization-invariant practical optimizer for modern Transformers. It does show why geometry tied to the *function performed* by the model is a more principled target than geometry assigned solely to a list of entries.

§1.12 Typical activations and worst-case activations disagree

The spectral norm considers the worst input direction. But the training data do not necessarily explore every direction equally often.

Let x have finite second moment $C_x = \mathbb{E}[xx^\top]$, which is positive semidefinite. The expected squared output change caused by D is

$$\begin{aligned}\mathbb{E}\|Dx\|_2^2 &= \mathbb{E}[x^\top D^\top Dx] \\ &= \operatorname{tr}(D^\top D\,\mathbb{E}[xx^\top]) \\ &= \operatorname{tr}(DC_xD^\top) = \|DC_x^{1/2}\|_{\mathrm{F}}^2. \end{aligned} \tag{1.14}$$

This is an exact identity, with no Gaussian assumption. If the mean of x is nonzero, C_x is the *second-moment* matrix; it is not the centered covariance unless the mean is zero.

For a numerical example, suppose

$$C_x = \begin{pmatrix} 100 & 0 \\ 0 & 1 \end{pmatrix}, \qquad D_1 = \varepsilon\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \qquad D_2 = \varepsilon\begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}.$$

Both updates have operator norm $|\varepsilon|$ and Frobenius norm $|\varepsilon|$. Yet

$$\mathbb{E}\|D_1x\|_2^2 = 100\varepsilon^2, \qquad \mathbb{E}\|D_2x\|_2^2 = \varepsilon^2.$$

A unit update in a frequently excited input direction can be a hundred times as consequential, in expected squared output change, as the same-sized update in a quiet direction.

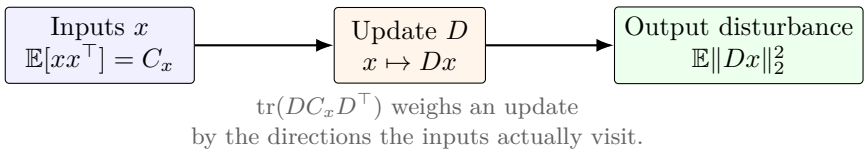


Figure 1.3: An input-dependent geometry counts how the update acts on the observed activation distribution. Operator norm instead bounds the most amplified direction, whether or not that direction is common.

If $C_x = I$, the data-weighted norm reduces to Frobenius norm. If C_x is strongly anisotropic, an unweighted Frobenius ball can be a poor model of functional change. The spectral norm remains valuable as a worst-case bound, but the example explains why it should not automatically be called the unique natural geometry of training.

§1.13 From change in activations to change in loss

Even equal output disturbance need not mean equal *loss* disturbance. Some output directions are much more consequential than others.

Let a layer's output be $y = Wx$, and suppose that, for a particular input x, a smooth local loss as a function of y has Hessian $H_y(x)$. Its second-order response to D contains the term

$$\frac{1}{2}\mathbb{E}_x\left[(Dx)^\top H_y(x)(Dx)\right]. \tag{1.15}$$

An arbitrary Hessian need not be positive semidefinite; a positive metric is obtained only when one uses an appropriate positive-semidefinite curvature surrogate, such as a Gauss–Newton or Fisher-type matrix.

In the instructive case where a fixed positive-definite output metric A is appropriate and the input second moment is $B = C_x$, the associated quadratic size becomes

$$\|D\|_{A,B}^2 = \operatorname{tr}(ADBD^\top) = \|A^{1/2}DB^{1/2}\|_{\mathrm{F}}^2. \tag{1.16}$$

The input side accounts for *where the network is used*; the output side accounts for *which changes matter*. This is a concrete reason to consider geometry on *both* sides of a weight matrix.

There is a familiar connection to natural gradient. A Fisher quadratic form measures the local change in a model's predictive distribution, often expressed through a second-order approximation to a Kullback–Leibler divergence. Kronecker-factored approximations in layered networks replace a large curvature matrix by smaller input and output factors. The product form (1.16) has the same algebraic shape.

But the factorization is an *approximation* in general: the exact expectation of $H_y(x) \otimes xx^\top$ need not equal the product of the expectations. A metric estimated from mini-batches may also be singular or noisy. None of these equations declares an optimizer optimal before its estimation cost and errors have been accounted for.

§1.14 The layers also compete for an update budget

Even after selecting a norm for each matrix, we must decide how to combine them. A network parameter space is a product:

$$\Theta = \mathbb{R}^{m_1\times n_1} \times \cdots \times \mathbb{R}^{m_L\times n_L} \times (\text{vector and scalar parameters}).$$

For simplicity, take just the matrix factors, with gradient G_ℓ at layer ℓ, and consider the weighted product norm

$$\|D\|_{\text{block}}^2 = \sum_{\ell=1}^{L} a_\ell \|D_\ell\|_{\text{op}}^2, \qquad a_\ell > 0.$$

The regularized first-order model is separable:

$$\min_{D_1,\dots,D_L} \sum_{\ell=1}^{L} \langle G_\ell, D_\ell\rangle + \frac{\lambda}{2}\sum_{\ell=1}^{L} a_\ell \|D_\ell\|_{\text{op}}^2. \tag{1.17}$$

Using the single-matrix duality result, a minimizer for every nonzero G_ℓ is

$$D_\ell = -\frac{\|G_\ell\|_*}{\lambda a_\ell}\ \operatorname{polar}(G_\ell). \tag{1.18}$$

In other words, a matrix's polar factor fixes its *direction*, while the weight a_ℓ and the layerwise nuclear norm decide *how much of the network budget* that matrix receives.

Compare this with a separate rule assigning each layer exactly the same scalar learning rate. Such a rule can use the same polar *directions*, yet it generally does not solve (1.17). With two layers whose gradient nuclear norms are 10 and 1, respectively, the first-order model with equal a_ℓ allots ten times the spectral step magnitude to the first layer. Equal fixed per-layer radii allot them equal magnitudes instead.

There is no theorem saying that proportional-to-gradient budgets will train an LLM best; it is an exact result for a particular local model. It exposes a choice that is easy to mistake for a mere implementation detail: *how are the norms of different modules aggregated?*

Recent studies make this issue concrete. Crawshaw and collaborators explore product norms and Muon variants with improved learning-rate robustness; He and collaborators demonstrate that learning rates adapted to layers' spectral statistics can help *both* Muon and AdamW. Better layerwise step allocation is therefore a serious alternative explanation for some improvements attributed entirely to the matrix orthogonalization operation.[3]

§1.15 A small singular value can carry loud noise

There is a second, less obvious difficulty with full spectral flattening: polar normalization may treat a tiny component as though it were as reliable as a large one.

Consider

$$G_\delta = \begin{pmatrix} 1 & 0 \\ 0 & \delta \end{pmatrix}, \qquad \delta \neq 0.$$

Its partial polar factor is

$$\operatorname{polar}(G_\delta) = \begin{pmatrix} 1 & 0 \\ 0 & \operatorname{sign}(\delta) \end{pmatrix}.$$

Changing δ from 10^{-6} to -10^{-6} changes the gradient matrix by only $2\cdot 10^{-6}$ in Frobenius norm. Yet it changes the polar factor by 2 in operator norm. As $\delta \to 0^{\pm}$, the two one-sided polar limits disagree.

This is an exact discontinuity in the ideal matrix sign at a rank change. It does *not* prove that a particular finite-step Muon implementation is unstable: the polynomial iteration, momentum, gradient noise distribution and finite precision all influence what happens. It does reveal an obvious potential cost of discarding the magnitude of every small singular component. A weak, unreliable singular mode may be raised to a full-sized update direction.

Partially compressing the spectrum, clipping a few large spectral spikes, or applying coordinatewise RMS normalization before spectral processing are different responses to this issue. They answer distinct questions: should we suppress dominant modes, preserve small ones, or estimate their reliability first?

[3] See Michael Crawshaw and collaborators, *An Exploration of Non-Euclidean Gradient Descent: Muon and its Many Variants* (ICML 2026), and Di He and collaborators, *One LR Doesn't Fit All* (ICML 2026).

§1.16 Why a larger first-order win can lose the actual step

A more dramatic example shows why the best *linearized* update need not be the best finite update. Consider an exact quadratic loss in two parameters,

$$\ell(z_1, z_2) = 8z_1 + z_2 + \frac{1}{2}(z_1^2 + 100z_2^2), \qquad z = (0, 0).$$

The gradient at the origin is $g = (8, 1)$. Put these parameters on the diagonal of a 2×2 matrix; the two singular directions are then precisely the coordinate axes. The curvature of the second direction is a hundred times that of the first.

At an operator-norm step budget of $\eta = 0.1$, consider

$$D_{\text{polar}} = -0.1\,\text{diag}(1, 1), \qquad D_{\text{scaled}} = -0.1\,\text{diag}(1, 1/8).$$

Both have operator norm 0.1. The polar step wins on the first-order term:

$$\langle G, D_{\text{polar}} \rangle = -0.9, \qquad \langle G, D_{\text{scaled}} \rangle = -0.8125.$$

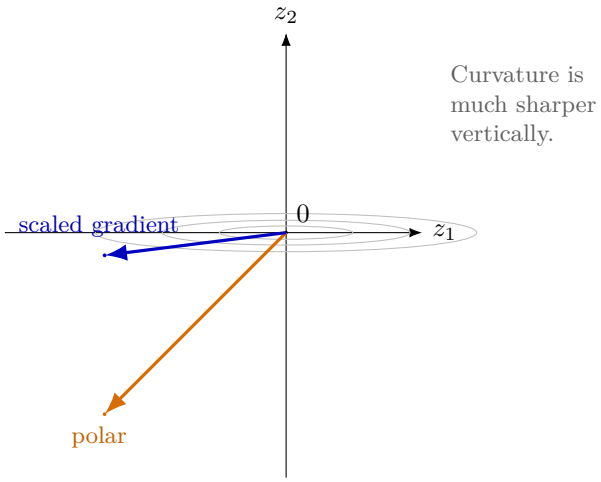


Figure 1.4: Two updates of equal spectral norm on the explicitly specified quadratic. Ellipses indicate the *curvature part* $\frac{1}{2}(z_1^2 + 100z_2^2)$, centered at the origin; the full loss has additional linear terms.

But the *actual* loss changes are

$$\ell(-0.1, -0.1) - \ell(0, 0) = -0.9 + \frac{1}{2}(0.01 + 100 \cdot 0.01) = -0.395,$$
$$\ell(-0.1, -0.0125) - \ell(0, 0) = -0.8125 + \frac{1}{2}(0.01 + 100 \cdot 0.0125^2) = -0.7996875.$$

The step with the *worse first-order prediction* lowers the true quadratic loss by roughly twice as much. Every number here comes from the stated quadratic; there is no neural-network experiment hidden inside the example.

What happened? The polar step spent much of its second-order curvature budget moving in the z_2 direction, where even a small displacement is expensive. In general,

$$L(W + D) - L(W) \approx \langle G, D \rangle + \frac{1}{2}\,\text{vec}(D)^\top H_W\,\text{vec}(D). \tag{1.19}$$

One can factor the second-order term as

$$\frac{1}{2}\|D\|_{\mathrm{F}}^2 \quad \underbrace{\frac{\operatorname{vec}(D)^\top H_W \operatorname{vec}(D)}{\|D\|_{\mathrm{F}}^2}}_{\text{directional curvature, for } D\neq 0} .$$

Thus a method can make a favorable first-order step, yet incur a large curvature penalty. The Hessian need not be positive in general; here it is positive definite, so the mechanism is particularly clear.

Shuche Wang and collaborators report smaller normalized directional sharpness for Muon than for Adam in their studied language-model settings.[4] That finding makes a plausible bridge from spectral update geometry to faster training, but it is an empirical and model-dependent observation. The quadratic above explains why such a curvature comparison is worth making, and why one should not infer a universally superior optimizer solely from the matrix sign formula.

§1.17 Early speed, eventual loss, and model quality

Several different empirical statements tend to be rolled together when discussing new optimizers. We should pull them apart.

First, reaching a validation-loss threshold in fewer tokens is a claim about data or optimization-step efficiency. *Second*, reaching it in less wall-clock time also depends on kernel speed, precision, communication and sharding. *Third*, reaching a smaller final loss depends on the training horizon, learning-rate schedule, weight decay, regularization and tuning budget. *Fourth*, a model's useful capabilities may not be accurately ordered by a small difference in aggregate next-token loss.

The published record illustrates these distinctions:

- In the large-scale Moonlight study, Muon was made practical with weight decay and matrix-shape-aware scaling, and showed strong training-efficiency results. Those findings concern a particular distributed implementation and training recipe.
- Wen, Hall, Ma and Liang compared optimizers across several language-model scales and training horizons with more systematic hyperparameter searches. They found that rankings can change between intermediate and final checkpoints; the advantage over tuned AdamW narrows at larger model sizes in their evaluated regime.
- Qi and collaborators evaluated a family $U\Sigma^a V^\top$ on a controlled 124M-parameter GPT-2 setup. In that setting, the completely flattened momentum update was not consistently best. RMS-normalized inputs and partial spectral compression were competitive, and the amount of compression mattered.
- Layerwise learning-rate studies and product-norm studies show that a large part of the optimization problem lies *outside* the normalization of any one matrix: how the method distributes step sizes over heterogeneous layers changes the comparison.

These observations are compatible. A method may win on the time needed to reach an early loss threshold, show a narrow final loss advantage under a different budget, and yield models whose downstream accuracies are statistically indistinguishable on the tested tasks. A good comparison must say which outcome matters and hold the other resources approximately fixed.

[4] See *Why Muon Outperforms Adam: A Curvature Perspective* (2026 preprint).

In particular, tuning one AdamW learning rate globally and giving Muon separate matrix/vector learning rates is an unfair test of the full recipes. But assigning separate learning rates to *every* AdamW layer also costs tuning resources, and Muon may benefit from the same freedom. A credible comparison reports its search space, search cost and held-out selection rule.[5]

§1.18 A more meaningful matrix geometry

We can now make a constructive guess. If the input statistics B and output sensitivity A matter, perhaps a useful update norm should be placed on

$$Z = A^{1/2} D B^{1/2}$$

rather than on the raw entries of D.

This suggestion leads to two precise first-order models. Suppose for the moment that $A \in \mathbb{R}^{m\times m}$ and $B \in \mathbb{R}^{n\times n}$ are symmetric positive definite and held fixed during the step.

A data-weighted quadratic budget. Consider

$$\min_D \left\{ \langle G, D\rangle + \frac{\lambda}{2} \|A^{1/2} D B^{1/2}\|_{\mathrm{F}}^2 \right\}. \tag{1.20}$$

Differentiating with respect to D gives

$$G + \lambda A D B = 0.$$

Hence

$$D_{\mathrm{quad}} = -\frac{1}{\lambda} A^{-1} G B^{-1}. \tag{1.21}$$

This is a two-sided preconditioning rule, not a mysterious new optimizer. It follows directly from the geometry of the chosen quadratic form.

A data-weighted spectral budget. Alternatively, constrain the worst-case singular gain of the *whitened* update:

$$\|A^{1/2} D B^{1/2}\|_{\mathrm{op}} \le \eta. \tag{1.22}$$

Set $Z = A^{1/2} D B^{1/2}$. Cyclicity of the trace gives

$$\langle G, D\rangle = \langle A^{-1/2} G B^{-1/2}, Z\rangle.$$

The spectral-norm result proved earlier applies immediately:

$$D_{\mathrm{wsp}} = -\eta A^{-1/2} \operatorname{polar}(A^{-1/2} G B^{-1/2}) B^{-1/2}. \tag{1.23}$$

These two formulas connect an Adam-like idea—use statistics to change the coordinates in which an update is measured—with a Muon-like idea—control the singular gains of the update as a map. They also show why a universal choice between Frobenius and spectral is too crude: *which input and output metrics* are used is part of the question.

The derivations are exact for fixed positive-definite A, B. A real implementation would need to estimate and regularize these factors, handle changing activation distributions, exploit block structure, distribute the extra state, and approximate inverse square roots. If those costs outweigh the benefit, a simpler optimizer may still be preferable.

[5] For controlled optimizer comparisons, see Kaiyue Wen, David Hall, Tengyu Ma and Percy Liang, *Fantastic Pretraining Optimizers and Where to Find Them* (ICLR 2026), and Xianbiao Qi and collaborators, *Delving into Muon and Beyond* (ICML 2026). Their setups and conclusions should not be collapsed into a single universal speedup.

This view puts several existing research directions in perspective. Schatten interpolation trades off preservation and flattening of singular values; spectral clipping can impose operator-norm control on the updates of other optimizers; joint constraints on *weights and updates* can limit activation drift. These are different ways of injecting matrix geometry into a practical training procedure, with different computational costs and guarantees.[6]

§1.19 How would we tell which geometry fits an LLM?

There is an experimental way to make the mathematics answerable. Consider a single reproducible Transformer architecture and a fixed training corpus. Compare AdamW, matrix/vector-decoupled AdamW, a properly tuned layerwise AdamW, Muon with the same access to layerwise tuning, and at least one intermediate spectral-power variant. The parameter groups, weight decay rules, data order, precision and random seeds must be stated.

For each recipe, distinguish three budgets: *tokens*, *wall-clock time* and *tuning computation*. A fourth resource, *peak GPU memory*, is especially important when the optimizer is being considered because of its state size. Record entire learning curves and final checkpoints instead of selecting only an early interval where one optimizer leads.

The geometry itself admits more informative probes than loss alone. On representative layers, measure the singular values of the gradient momentum and the *actual applied updates*; the ratio between update RMS and weight RMS; the distribution of activation second moments; and the layerwise output changes Dx induced by each update. If curvature-vector products are affordable, estimate directional curvature along the actual optimizer steps. This helps separate three possible explanations:

1. spectral flattening changes which singular directions are explored;
2. shape and layerwise learning-rate rules change where the update budget is spent;
3. the resulting steps align better with the local curvature or with activation directions that matter.

These explanations are not mutually exclusive. A method can save optimizer-state memory independently of all three, and a lower final validation loss remains only one measurement of the trained model.

A particularly diagnostic control is to keep *the same polar direction* but vary only the layerwise scaling, then to keep *the same layerwise scaling* while varying spectral compression. The two comparisons isolate effects that a single Muon-versus-AdamW curve otherwise mixes together.

§1.20 Returning to the first two singular values

The matrix $\mathrm{diag}(8, 1)$ gave us a choice between following a strong gradient mode and distributing our effort over two modes. With a spectral-norm budget and a linearized loss, the polar step wins; that is a theorem. With a nontrivial curvature, the same step can lose at finite learning rates; we exhibited an exact quadratic. With anisotropic

[6] See Xiaowen Jiang, Andrei Semenov and Sebastian Stich, *Enhancing LLM Training via Spectral Clipping* (ICML 2026), and Tian Xie and collaborators, *Controlled LLM Training on Spectral Sphere* (2026 preprint). The second work proposes constraints on both parameters and their updates; its empirical scope should be kept distinct from a general theorem of optimality.

activations, equal-sized matrix steps can change the network's outputs by very different amounts; this too followed from a one-line trace identity.

The original practical observations now fit into a sharper picture. Muon genuinely reduces persistent state for the hidden matrices to which it applies. Its spectral update is often an effective way to spend a matrix-shaped step, and relatively little machinery is required to implement it efficiently on modern accelerators. Its early loss advantage, when observed, does not settle which geometry would be best for every module or throughout training. Similar end-of-run losses need not imply identical optimization dynamics, and small differences in aggregate loss need not imply a materially better downstream model.

If there is a particularly natural geometry for language-model training, it must respect at least *what the matrices do*, *how their inputs are distributed*, *what their outputs influence*, and *how step sizes are coordinated across the network*. Spectral norm is a strong and elegantly motivated starting point. The remaining mathematical task is to understand when its worst-case notion of size captures the perturbations a Transformer actually cares about.